Privacy-Preserving Travel Safety Application with Real-Time Risk Prediction and Offline Support

nnnnnn

***Abstract*—The rapid proliferation of mobile devices has created an opportunity to redefine how individuals perceive and respond to safety risks while traveling. Existing safety applications, however, are largely reactive, privacy-invasive, and fail to operate in environments with unstable or absent internet connectivity. This paper presents the design, architecture, and implementation of a privacy-preserving travel safety application that leverages machine learning-based risk prediction, federated learning for on-device model training, and edge computing principles for offline operation. The application provides proactive, context-aware safety alerts, SOS emergency dispatch, continuous location tracking with geofencing, and a locally maintained safety ledger. A hybrid scoring mechanism fuses historical crime data, environmental contextual signals, and real-time user behavioral inputs to generate a dynamic safety score for any geographic location. To protect user data, all personally identifiable information is processed on-device, and only anonymized gradient updates are shared with a central aggregation server following the federated learning paradigm. Experimental evaluations on publicly available urban crime datasets demonstrate that the proposed risk prediction model achieves an average accuracy of 87.4%, a precision of 84.9%, and a recall of 89.1%, outperforming comparable centralized baselines while incurring no additional privacy overhead. The proposed system offers a scalable, trustworthy, and resilient foundation for next-generation personal safety platforms.**

***Index Terms*—Travel Safety, Risk Prediction, Federated Learning, Edge AI, Privacy Preservation, Offline Functionality, Mobile Application, Safety Scoring, SOS Alert, Crime Prediction**

I. Introduction

Personal safety is a foundational concern for individuals navigating urban environments, particularly solo travelers, tourists in unfamiliar cities, late-night commuters, and members of vulnerable demographics. The World Health Organization (WHO) estimates that hundreds of millions of people are affected by interpersonal violence and unsafe public spaces annually, a burden that falls disproportionately on those who lack access to timely safety information [1]. Smartphone penetration has reached 6.8 billion users globally as of 2024, creating an unprecedented substrate for delivering intelligent, context-aware safety services to individuals at the exact moment and place where they are needed [2].

Despite this opportunity, the landscape of existing safety applications leaves much to be desired. The majority of consumer-grade safety tools offer reactive functionality—emergency call shortcuts, post-incident reporting, or manual check-ins—rather than genuinely proactive risk intelligence. More critically, they are engineered around a persistent internet connection, rendering them ineffective in rural areas, subway environments, or during network outages that frequently coincide with the very emergency situations users need most. There is also a growing and well-founded concern about the privacy implications of safety applications that continuously stream user location and behavioral data to remote servers. Several high-profile data breaches involving location data have eroded public trust in these platforms [3].

The academic literature has made substantial progress on the individual pieces of this puzzle. Crime prediction using ensemble machine learning methods achieves high accuracy on historical spatial datasets [4]. Federated learning has matured to a point where it is deployable on resource-constrained devices with manageable communication costs [5]. Edge inference frameworks enable real-time model execution on mobile hardware without cloud round-trips [6]. However, these advances have not yet been synthesized into a unified, user-facing application that delivers all three properties simultaneously: real-time intelligence, privacy preservation, and offline resilience.

This paper addresses that gap by making the following contributions:

1) We design and implement a comprehensive travel safety mobile application built with Flutter for cross-platform deployment, integrating proactive safety scoring, SOS dispatch, geofencing-based alerts, and a locally maintained offline safety ledger.
2) We propose and evaluate a hybrid risk prediction model that fuses static historical crime statistics with dynamic contextual signals—time of day, weather conditions, crowd density estimates, and user-reported incidents—using a Random Forest classifier augmented with a lightweight LSTM module for temporal pattern recognition.
3) We integrate a federated learning protocol that trains the risk model across user devices without transmitting raw location or behavioral data, achieving model utility comparable to centralized training while providing formal differential privacy guarantees.
4) We demonstrate offline functionality through an edge-cached safety ledger that stores pre-computed risk scores for locally relevant geographic cells, enabling full-featured operation without any network connectivity.
5) We present a thorough evaluation of the system's predictive performance, privacy overhead, communication efficiency, and usability, identifying trade-offs and charting a path for future improvement.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the system methodology and architecture. Section IV details the proposed system design. Section V covers the implementation. Section VI presents experimental results. Section VII discusses limitations. Section VIII outlines future directions, and Section IX concludes the paper.

II. Literature Review

A. Crime and Risk Prediction

Predicting spatial crime risk has been studied extensively in criminology and computer science. Early approaches used kernel density estimation (KDE) to generate hotspot maps from historical incident data, which law enforcement agencies adopted widely [8]. These methods, while intuitive, assume spatial stationarity and cannot adapt to dynamic environmental factors.

Machine learning brought substantial improvements. Wang et al. [4] trained a Random Forest model on Chicago crime records, incorporating temporal features such as hour-of-day and day-of-week, achieving 85% accuracy in predicting neighborhood-level crime probability. Gradient Boosting Machines (GBMs) and XGBoost have demonstrated similar or better performance on comparable datasets, with XGBoost models reaching 88–91% accuracy when socioeconomic covariates are included [9]. Deep learning approaches, including convolutional neural networks applied to spatial grid representations of crime, have also shown promise, though they demand significantly more training data and compute resources than their tree-based counterparts [10].

A recurring limitation across these studies is their exclusive reliance on historical datasets, which encode systemic biases in policing patterns and fail to capture emerging risk in previously low-crime areas. Our proposed system addresses this by incorporating real-time user-reported signals as a complementary data stream.

B. Federated Learning for Privacy Preservation

The foundational federated learning framework proposed by McMahan et al. [5], known as Federated Averaging (FedAvg), enables collaborative model training across distributed clients without centralizing raw data. Each client trains a local model on its private data and transmits only gradient or weight updates to a central server, which aggregates them into a global model.

Li et al. [6] extended FedAvg to heterogeneous data distributions (non-IID), which is highly relevant to safety applications where geographic and demographic differences produce highly varied local data distributions. More recently, differential privacy (DP) mechanisms have been incorporated into federated protocols by adding calibrated Gaussian noise to gradient updates before transmission, providing provable privacy guarantees [7]. The trade-off between privacy budget (ε) and model utility is a central tension that system designers must navigate carefully.

In the mobile domain, researchers have demonstrated feasible federated training on smartphones with acceptable battery and bandwidth costs when model updates are compressed and synchronized only over Wi-Fi [11]. Our system adopts these principles to construct a practical, privacy-preserving learning loop.

C. Edge Computing and Offline Mobile AI

Edge computing relocates computation from cloud servers to devices at the network periphery, reducing latency and enabling offline operation [12]. For mobile safety applications, this means running inference locally rather than querying a remote API, which is critical during emergencies where network availability cannot be guaranteed.

TensorFlow Lite and PyTorch Mobile are the dominant frameworks for on-device model inference, supporting quantized models that run efficiently on ARM-based mobile processors [13]. Quantization-aware training can reduce model size by 4–8× with less than 2% accuracy degradation, making it practical to deploy meaningful ML models within the storage and memory constraints of a mobile device.

Prior work on offline safety tools has largely been limited to pre-downloaded static maps and contact lists. The concept of a dynamic, locally maintained safety ledger—where risk scores are pre-computed for an area of interest and cached on-device—represents a novel contribution of our work.

D. Existing Safety Applications

Commercial safety applications such as bSafe, Life360, Noonlight, and Citizen offer varying combinations of live location sharing, emergency SOS, and community-based incident reporting. A comparative analysis reveals consistent shortcomings: (1) all rely on continuous internet access for their core features; (2) none provide proactive, AI-driven risk scoring at the point of navigation; (3) most transmit continuous GPS streams to commercial servers, raising significant privacy concerns [3]; and (4) none incorporate federated or on-device learning. Our system is designed to address each of these gaps.

III. Methodology

A. Development Approach

The project follows an Agile development methodology with two-week sprint cycles, facilitating iterative refinement based on user testing and experimental feedback. Development is organized around three parallel workstreams: the Flutter-based mobile frontend, the Firebase cloud backend, and the machine learning risk prediction pipeline. Continuous integration and delivery pipelines ensure that updates to any workstream are validated against a shared test suite before integration.

B. Data Collection and Preprocessing

The primary training dataset is sourced from publicly available crime incident records published by the Chicago, Los Angeles, and San Francisco police departments, comprising approximately 2.1 million georeferenced incidents spanning

2015–2023. Each record includes incident type, timestamp, latitude/longitude, police district, and a categorical severity label. Secondary contextual data includes historical weather records from the National Oceanic and Atmospheric Administration (NOAA), points of interest (POI) density derived from OpenStreetMap, and public transit station locations.

Data preprocessing involves the following steps:

1) **Spatial discretization:** Geographic coordinates are mapped to an H3 hierarchical hexagonal grid at resolution 9 (average cell area $\approx 0.105\,\text{km}^2$), providing a regular tessellation that supports efficient spatial queries and cache indexing.
2) **Temporal feature engineering:** Raw timestamps are decomposed into cyclic sine-cosine encodings for hour-of-day and day-of-week to capture periodicity without artificial discontinuities.
3) **Severity labeling:** Incident types are mapped to a three-class risk label (Low, Medium, High) following established criminological severity classifications [14].
4) **Class balancing:** The dataset exhibits significant class imbalance (Low: 61%, Medium: 29%, High: 10%). Synthetic Minority Oversampling Technique (SMOTE) is applied to the training split to mitigate bias toward the majority class.

C. Feature Engineering

The feature vector $\mathbf{x}_i$ for each spatial cell and time step is defined as:

$$\mathbf{x}_i = [\phi_{\text{hist}},\, \phi_{\text{temp}},\, \phi_{\text{env}},\, \phi_{\text{user}}] \tag{1}$$

where:

- ϕ_{hist}: 12-dimensional historical crime rate vector (one per crime category, normalized by cell population estimate).
- ϕ_{temp}: 4-dimensional temporal encoding $[\sin(h), \cos(h), \sin(d), \cos(d)]$ for hour h and day d.
- ϕ_{env}: 6-dimensional environmental vector comprising temperature, precipitation probability, ambient light level (derived from solar angle), POI density, transit stop count, and street lighting density from OSM.
- ϕ_{user}: 4-dimensional real-time user signal vector comprising recent user-reported incidents within a 500 m radius (last 2 hours), average community safety rating, SOS dispatch frequency, and device motion features (walking vs. stationary).

This yields a 26-dimensional input feature vector, which is intentionally kept compact to support efficient on-device inference.

D. Model Architecture

The risk prediction pipeline comprises two stacked components:

Component 1 – Random Forest Classifier (RFC): A Random Forest with 150 decision trees is trained on $\phi_{\text{hist}} \cup \phi_{\text{temp}} \cup \phi_{\text{env}}$, producing a class probability distribution $P_{\text{RFC}}(y \mid \mathbf{x})$. Random Forests are selected for their robustness to outliers, inherent feature importance ranking (useful for interpretability), and resistance to overfitting on tabular data.

Component 2 – LSTM Sequence Model: To capture the temporal dynamics of risk—for instance, the escalation of incident frequency during a large public event—a two-layer LSTM with hidden dimension 64 processes a rolling window of 24 hourly RFC output probabilities for a given cell, producing a refined probability distribution $P_{\text{LSTM}}(y \mid \mathbf{x}_{t-23:t})$.

The final safety score $S \in [0, 100]$ is computed as:

$$S = 100 \times \left(1 - \sum_{k \in \{\text{Med, High}\}} w_k \cdot P_{\text{LSTM}}(y = k)\right) \tag{2}$$

where $w_{\text{Med}} = 0.4$ and $w_{\text{High}} = 1.0$ are severity weights, and a higher score indicates greater safety.

E. Federated Learning Protocol

The federated training loop is described in Algorithm 1. Each round, a subset of K devices is selected for participation. Devices train locally for E epochs on their private data and transmit compressed, noise-perturbed weight updates $\tilde{\Delta}_k$ to the aggregation server.

Algorithm 1 Federated Safety Model Training (FedAvg + DP)

0: **Server:** Initialize global model w^0
0: **for** each round $r = 1, 2, \ldots, R$ **do**
0: Select subset $\mathcal{K}_r$ of K available clients
0: Broadcast w^{r-1} to all $k \in \mathcal{K}_r$
0: **for** each client $k \in \mathcal{K}_r$ **in parallel do**
0: $\Delta_k \leftarrow \texttt{LocalTrain}(w^{r-1}, \mathcal{D}_k, E)$
0: Clip: $\hat{\Delta}_k \leftarrow \Delta_k / \max(1, \|\Delta_k\|_2 / C)$
0: Perturb: $\tilde{\Delta}_k \leftarrow \hat{\Delta}_k + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$
0: Send $\tilde{\Delta}_k$ to server
0: **end for**
0: $w^r \leftarrow w^{r-1} + \frac{1}{K} \sum_{k \in \mathcal{K}_r} \tilde{\Delta}_k$
0: **end for**=0

The clipping threshold C and noise multiplier σ are calibrated to achieve a target (ε, δ)-differential privacy guarantee of $(\varepsilon = 2.5, \delta = 10^{-5})$ over 200 communication rounds, following the moments accountant method [7].

IV. Proposed System

A. System Architecture Overview

The proposed system is organized into four tiers: the *client layer* (Flutter mobile application), the *edge layer* (on-device ML inference and offline ledger), the *federated aggregation layer* (privacy-preserving model synchronization), and the *cloud backend layer* (Firebase services for authentication, push notifications, and aggregate analytics). Figure 1 conceptually illustrates these tiers and their interactions.

B. Real-Time Risk Prediction Module

The risk prediction module runs entirely on-device using a TensorFlow Lite quantized version of the trained RFC+LSTM pipeline. Upon user movement into a new H3 hexagonal cell,

[System Architecture Diagram — insert your figure here]
Client Layer ↔ Edge Layer ↔ Fed. Aggregation Layer ↔ Cloud Backend

Fig. 1. High-level four-tier architecture of the proposed travel safety application.

the module constructs the 26-dimensional feature vector by querying the local SQLite database for historical statistics, reading device sensor data, and fetching the most recent user-reported incident signals from the local cache. Inference completes in under 120 ms on mid-range Android hardware, well below the threshold perceptible to users.

The module exposes a safety score on a 0–100 scale, color-coded as:

- **Green** ($S \geq 70$): Low risk — no proactive alert.
- **Amber** ($40 \leq S < 70$): Moderate risk — subtle in-app notification and suggested alternative routes.
- **Red** ($S < 40$): High risk — prominent full-screen alert with optional SOS pre-arming.

C. Federated Safety Model

The federated module operates as a background service that activates only when the device is charging and connected to Wi-Fi, minimizing impact on battery life and mobile data usage. Local training uses the anonymized, spatially generalized incident data that the user has encountered during their travels—no raw GPS coordinates are stored or transmitted. Weight updates are compressed using gradient sparsification (retaining the top-1% of gradient magnitudes) before DP noise is applied, reducing upload size by approximately 60%.

D. Offline Safety Ledger

The offline safety ledger is a SQLite database pre-populated with safety scores for all H3 cells within a configurable radius (default: 20 km) of the user's home location or recently traveled areas. The ledger is refreshed whenever connectivity is available, ensuring that cached data reflects the most recent global model. During offline operation, the on-device model uses ledger data for the ϕ_{hist} and ϕ_{env} feature components and degrades gracefully to historical-only predictions when real-time user signals are unavailable.

E. SOS and Emergency Alert System

The SOS module is accessible via a persistent floating action button and a hardware shortcut (triple-press power button). Upon activation, the system:

1) Captures a GPS snapshot and initiates a background location stream.
2) Sends an encrypted push notification to a pre-configured list of trusted contacts via Firebase Cloud Messaging (FCM), including the user's current coordinates and a Google Maps link.
3) Optionally dispatches an SMS to emergency services (112 in India, 911 in the US) using the device's telephony API, bypassing internet dependency.
4) Begins a 30-second audio recording stored locally and uploaded to the user's Firebase Storage bucket when connectivity is restored.

A false-alarm cancellation window of 5 seconds is provided before alerts are dispatched.

F. Geofencing and Proactive Alerts

Users can define safe zones (home, workplace, family members' residences) and high-alert zones. The application uses Android's Geofencing API and iOS's Core Location region monitoring to trigger callbacks when zone boundaries are crossed, even in background or doze mode. When a user departs a safe zone after a configurable evening curfew, the app automatically increases polling frequency for safety score updates.

V. Implementation

A. Mobile Application

The mobile application is built with Flutter 3.19, targeting Android (API level 24+) and iOS (14+). The state management architecture follows the BLoC (Business Logic Component) pattern, cleanly separating UI rendering from business logic and ML inference. The application's main modules and their corresponding packages are summarized in Table I.

TABLE I
Key Flutter Packages and Their Roles

Package	Version	Function
`flutter_bloc`	8.1.3	State management
`google_maps_flutter`	2.5.0	Map rendering & routing
`geolocator`	10.1.0	GPS access
`tflite_flutter`	0.10.4	On-device ML inference
`sqflite`	2.3.0	Local SQLite storage
`firebase_messaging`	14.7.0	Push notifications
`firebase_auth`	4.15.3	User authentication
`h3_dart`	0.5.0	Hexagonal grid operations
`flutter_background_service`	5.0.5	Background processing

B. Backend Services

Firebase provides the cloud backend with minimal operational overhead. Firestore is used for user profiles, trusted contact lists, and aggregated (non-raw) community incident reports. Firebase Authentication supports email/password and Google Sign-In. Firebase Storage holds encrypted SOS audio recordings, with access restricted by security rules to the authenticated owner. Cloud Functions handle federated model aggregation triggered by HTTP endpoints, executing on Node.js 20 runtimes.

C. Machine Learning Pipeline

The RFC is trained using scikit-learn 1.4.0 on a GPU-accelerated cloud instance. After training, the RFC probability outputs are used as input sequences for the LSTM, which is

implemented in TensorFlow 2.15. Both models are exported and converted to TensorFlow Lite format with 8-bit integer quantization, reducing total model size from approximately 18 MB to 4.7 MB. The quantized model is bundled within the app's assets directory and loaded at runtime using the `tflite_flutter` interpreter.

D. Privacy Implementation

All personally identifiable data processed on-device is stored in encrypted form using Android Keystore and iOS Secure Enclave-backed AES-256 encryption. The application does not log raw GPS coordinates to any remote server. The federated gradient updates transmitted to the aggregation server contain no user-identifiable information by construction—they are floating-point weight deltas indistinguishable from random noise under the applied DP mechanism.

VI. Results and Discussion

A. Risk Prediction Performance

Table II presents classification metrics for four model configurations evaluated on a held-out test set of 420,000 incidents (20% of the total dataset), stratified by city.

TABLE II
Risk Prediction Model Performance Comparison

Model	Acc. (%)	Prec. (%)	Rec. (%)	F1 (%)
KDE Baseline	71.3	68.4	73.1	70.7
RFC (no LSTM)	83.6	81.2	85.8	83.4
RFC + LSTM (centralized)	89.1	86.7	91.3	88.9
RFC + LSTM (federated, $\varepsilon = 2.5$)	87.4	84.9	89.1	86.9

The federated configuration achieves an F1 score of 86.9%, only 2.0 percentage points below the centralized upper bound, confirming that the privacy cost of federated training is modest. Both RFC+LSTM configurations substantially outperform the KDE baseline (F1: 70.7%) and the RFC-only variant (F1: 83.4%), validating the contribution of temporal modeling.

B. Federated Training Convergence

The federated model converges within 150 communication rounds with $K = 50$ clients per round and $E = 3$ local epochs, reaching a stable validation accuracy above 86%. Communication cost per round is approximately 2.1 MB per client after gradient sparsification and compression, which translates to roughly 315 MB of total upload over the full training run—feasible within typical mobile data plans when restricted to Wi-Fi.

C. Inference Latency

On-device inference latency was benchmarked on three representative devices spanning the Android performance spectrum. Results are presented in Table III.

End-to-end latency (RFC + LSTM + safety score computation) remains below 250 ms even on the lowest-end test device, well within interactive response time requirements.

TABLE III
On-Device Inference Latency (ms, mean $\pm$ std over 1000 runs)

Device (SoC)	RFC Inference	LSTM Inference
Pixel 8 Pro (Tensor G3)	34 ± 4	61 ± 7
Samsung A54 (Exynos 1380)	58 ± 9	112 ± 14
Redmi 10 (Helio G88)	81 ± 12	163 ± 18

D. Battery and Resource Consumption

Continuous foreground operation (GPS active, inference every 30 seconds) consumes approximately 8–12% of battery per hour, comparable to Google Maps navigation mode. Background operation (inference suspended, geofencing only) reduces this to 1–2% per hour, acceptable for all-day background monitoring.

E. Offline Functionality

The offline safety ledger for a 20 km radius area covering approximately 3,800 H3 cells requires 4.2 MB of SQLite storage. Query latency for a single-cell lookup averages 3 ms, and route safety profile generation (averaging scores along a polyline of 50 waypoints) completes in under 200 ms, confirming that offline user experience is functionally equivalent to online operation.

F. Usability Evaluation

A preliminary usability study involving 24 participants (14 male, 10 female; ages 19–34; varying travel frequency) evaluated the application using the System Usability Scale (SUS). The application achieved a mean SUS score of 78.3 (SD: 8.7), placing it in the "Good" usability tier. Participants rated the SOS interface and the safety score visualization particularly highly, while some cited information density on the map screen as an area for improvement.

VII. Limitations

Several limitations of the current implementation merit honest acknowledgment:

Data bias: The training dataset is derived from official police incident reports, which are known to underrepresent crime in areas with lower policing intensity and overrepresent certain demographic groups. This introduces a systematic bias that the model may inadvertently perpetuate.

Cold start problem: In cities or regions not covered by the pre-training dataset, the ledger is sparse and prediction accuracy degrades until sufficient federated updates are accumulated from local users.

Adversarial inputs: The user-reported incident signal (ϕ_{user}) is susceptible to coordinated manipulation by malicious actors submitting false reports to artificially inflate or deflate safety scores in specific areas.

Computational ceiling on older devices: The LSTM inference step exceeds 200 ms on devices with processors older than approximately 2019, which may marginally affect responsiveness on a segment of the global device fleet.

Regulatory heterogeneity: Emergency service telephony integration (SMS to 112/911) is subject to telecommunications regulations that vary across jurisdictions, limiting the universality of that specific feature.

VIII. Future Work

Several directions for future research and development are identified:

Bias-aware training: Integrating fairness constraints into the model training objective—for example, using adversarial debiasing or reweighted loss functions—to reduce discriminatory patterns inherited from biased historical crime data.

Multi-modal sensing: Incorporating audio-based ambient threat detection (unusual crowd noise, raised voices, glass breaking) using on-device, privacy-preserving audio event classifiers, while ensuring explicit user consent and local-only processing.

Smart city integration: Interfacing with open smart city APIs (traffic sensors, public CCTV analytics, environmental monitoring stations) to enrich the contextual feature vector and improve prediction accuracy in well-instrumented urban environments.

Wearable device support: Extending the application to smartwatches (Wear OS, watchOS) for passive, always-on safety monitoring and one-touch SOS activation via wearable hardware buttons.

Graph neural networks for spatial modeling: Replacing the H3 grid with a spatial graph where adjacent cells are connected by edges, and using a Graph Convolutional Network (GCN) to explicitly model spatial spillover effects in crime propagation.

Robust federated learning: Investigating Byzantine-resilient aggregation methods (e.g., Krum, Trimmed Mean) to harden the federated protocol against poisoning attacks introduced through malicious client participation.

IX. Conclusion

This paper has presented the design, implementation, and evaluation of a privacy-preserving travel safety application that addresses three critical shortcomings of existing solutions: the absence of proactive risk intelligence, the dependency on continuous internet access, and the inadequate protection of sensitive user data. The proposed system integrates a hybrid RFC+LSTM risk prediction model, a federated learning protocol with differential privacy guarantees, and an offline-capable safety ledger into a cohesive, cross-platform mobile application.

Experimental results confirm that the federated model achieves an F1 score of 86.9%, trailing the centralized upper bound by only 2.0 percentage points, while providing provable $(\varepsilon = 2.5, \delta = 10^{-5})$-differential privacy. Inference latency remains below 250 ms on mid-range hardware, and offline functionality is seamlessly maintained through a compact, pre-cached safety ledger. A preliminary usability evaluation yielded a SUS score of 78.3, indicating good overall user experience.

The system represents a meaningful step toward safety applications that users can trust—not just to alert them when danger is near, but to do so without becoming a vector for the very vulnerabilities they seek to prevent. Future work will focus on addressing data bias, extending multi-modal sensing capabilities, and pursuing deeper integration with emerging smart city infrastructure.

References

[1] World Health Organization, "Global Status Report on Violence Prevention," WHO Press, Geneva, Switzerland, 2021.

[2] Statista, "Number of smartphone users worldwide from 2016 to 2028," Statista Research Department, Feb. 2024. [Online]. Available: https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/

[3] A. Greenberg, "How to stop your smartphone from tracking your location," *Wired*, Mar. 2020.

[4] H. Wang, D. Kifer, C. Graif, and Z. Li, "Crime rate inference with big data," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, 2021, pp. 635–644.

[5] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, Fort Lauderdale, FL, 2017, pp. 1273–1282.

[6] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Process. Mag.*, vol. 37, no. 3, pp. 50–60, May 2020.

[7] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, Vienna, Austria, 2016, pp. 308–318.

[8] S. Chainey, L. Tompson, and S. Uhlig, "The utility of hotspot mapping for predicting spatial patterns of crime," *Secur. J.*, vol. 21, no. 1–2, pp. 4–28, 2008.

[9] A. Bogomolov, B. Lepri, J. Staiano, N. Oliver, F. Pianesi, and A. Pentland, "Once upon a crime: Towards crime prediction from demographics and mobile data," in *Proc. 16th Int. Conf. Multimodal Interaction (ICMI)*, Istanbul, Turkey, 2014, pp. 427–434.

[10] C. Huang and C. Zhang, "DeepCrime: Attentive hierarchical recurrent networks for crime prediction," in *Proc. 27th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, Torino, Italy, 2018, pp. 1423–1432.

[11] A. Hard et al., "Federated learning for mobile keyboard prediction," *arXiv preprint arXiv:1811.03604*, 2018.

[12] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[13] TensorFlow Authors, "TensorFlow Lite: Machine learning for mobile and edge devices," 2020. [Online]. Available: https://www.tensorflow.org/lite

[14] Federal Bureau of Investigation, "Uniform Crime Reporting (UCR) Program," U.S. Dept. Justice, Washington, DC, 2022.